

Bayesian Hidden Markov Models to Infer Latent Pitcher States

Garrett Lee, Trey Elder, Isaac Herman, Injun Lyo

1. Motivation

What Problem Does This Method Solve?

Why Can't Standard Regression Handle It?

Where Does It Show Up in Practice?

2. Model Specification

Likelihood

Priors

Hierarchical Structure

Key Modeling Assumptions

3. Applied Example

Data Description and Context

We apply a Bayesian Hidden Markov Model to pitch-level velocity data for **Gerri Cole**, a right-handed starting pitcher for the New York Yankees. Cole was selected for his high four-seam fastball usage rate and consistent full seasons of starts (30+ per year), giving us roughly 1,000–2,000 four-seam fastballs per season to work with.

Emission metric: We model a single observable: `release_speed` (pitch velocity in mph) for four-seam fastballs only. Restricting to one pitch type is essential, since mixing pitch types would cause the HMM to discover pitch type as the hidden state rather than performance condition. Velocity is the most straightforward and interpretable signal of a pitcher's condition. Elevated velocity indicates peak effort, while declining velocity often signals fatigue.

Hidden states: The model posits two latent states, *Baseline* and *Hot*, representing the pitcher's true performance mode at each pitch. The state is unobserved; we only see the noisy velocity measurements that each state generates.

Game structure: Pitches are grouped by game start, and each game is treated as an **independent sequence**. The hidden state resets at the beginning of each game. This reflects the reality that games are separated by 4–5 days of rest, so carry-over fatigue between games is not modeled.

Data source: Pitch-level data come from MLB Statcast via the `baseballr` package, covering the 2019–2024 seasons. The raw data are fetched week-by-week and cached locally to avoid repeated API calls.

Load Data

We pull pitch-level Statcast data for Gerrit Cole across the 2019–2024 MLB seasons. Data are retrieved in weekly chunks from Baseball Savant and saved locally so subsequent runs read from the cache rather than re-querying the API. All pitch types are retained at this stage; filtering to four-seam fastballs happens in the cleaning step.

Below is a sample of the raw data that was collected:

Pitcher	Date	Season	Pitch Type	Velocity (mph)	Spin Rate (rpm)
Cole, Gerrit	2019-04-03	2019	KC	83.5	3181
Cole, Gerrit	2019-04-03	2019	CH	87.1	1667
Cole, Gerrit	2019-04-03	2019	FF	97.9	2605
Cole, Gerrit	2019-04-03	2019	KC	83.8	2959
Cole, Gerrit	2019-04-03	2019	SL	89.4	2777
Cole, Gerrit	2019-04-03	2019	FF	98.8	2634

Clean Data

We filter the raw data to four-seam fastballs for the 2024 season and restructure it into the format expected by the HMM. Within each season, games are numbered sequentially by date. Within each game, pitches are sorted by inning, then at-bat, then position within the at-bat to recover true chronological order, and renumbered 1, 2, 3, ... as a single game-level sequence.

Season	Pitcher	Game	Pitch	Velocity (mph)
2024	Cole, Gerrit	1	1	94.2
2024	Cole, Gerrit	1	2	94.1
2024	Cole, Gerrit	1	3	97.0
2024	Cole, Gerrit	1	4	96.6
2024	Cole, Gerrit	1	5	95.7
2024	Cole, Gerrit	1	6	95.2

Fit Model

Prior Selection

We need to center the Normal prior on each emission mean somewhere reasonable. As a starting point, we use Cole’s historical velocity data from all seasons prior to 2024, split at the 75th percentile: the lower 75% of pitches approximate the Baseline state and the upper 25% approximate the Hot state. The mean of each group becomes the center of the corresponding prior. This is a rough heuristic. A 75/25 split doesn’t correspond to any structural assumption about how often Cole is in each state. The intent is simply to give the sampler a sensible starting region rather than an uninformative flat prior. With thousands of 2024 pitches available, the likelihood will dominate and the prior centers matter little in practice.

State	Mean (mph)	SD (mph)	N (pitches)
Baseline	96.71	1.03	5388
Hot	98.85	0.56	1756

The three sets of model parameters each receive their own prior.

For the **emission means** (μ_k), we use Normal priors centered on the empirical group means above, with a standard deviation of 2 mph. This is wide enough that the posterior is driven by the 2024 data, but anchored close enough to realistic velocity values that the sampler does not waste time exploring implausible regions.

State	Prior mean (mph)	Prior SD (mph)
Baseline	96	2
Hot	99	2

For the **transition matrix rows** (ν_k), we use Dirichlet priors that place more weight on self-transitions than on switching. A concentration parameter of 8 on the diagonal versus 1 on the off-diagonal reflects the prior belief that a pitcher’s state tends to persist from one pitch to the next — states are sticky, not erratic.

Current state	Stay (Dirichlet param)	Switch (Dirichlet param)	Prior P(stay)
Baseline	8	1	~89%
Hot	8	1	~89%

For the **initial state distribution** (π), we use a Dirichlet prior that puts most of its weight on starting in the Baseline state at the beginning of each game. The asymmetry reflects the reasonable assumption that a pitcher is more likely to begin a start in a typical rather than elevated performance mode.

State	Dirichlet param	Prior P(start here)
Baseline	6	~75%
Hot	2	~25%

Gibbs Sampler

The HMM has three sets of unknown parameters: the emission means μ , the transition matrix ν , and the initial state distribution π , plus the hidden state sequence itself. Because the full joint posterior over all of these is intractable, we use a **Gibbs sampler**: a Markov chain Monte Carlo algorithm that cycles through each block of unknowns in turn, drawing from its conditional distribution given the current values of everything else. After a burn-in period, the chain’s samples approximate the joint posterior.

The hidden state sequences are a fourth unknown, but they are not a parameter in the usual sense. They are latent variables that must be integrated over. Because conditioning on the state sequences makes the parameter updates simple conjugate draws, we include them in the sampler and alternate between updating states and updating parameters.

Each iteration of the sampler performs four steps:

Step 1: Sample hidden states (FFBS). Given the current parameter values, we sample a complete hidden state sequence for each game independently using the **Forward-Filter Backward-Sample** algorithm. In the forward pass, we compute $\alpha_{t,k}$, the joint probability of the observed velocities up through pitch t and being in state k at pitch t . This recursion sweeps forward through the game, accumulating evidence. In the backward pass, we draw states from right to left: the last pitch’s state is sampled from the normalized final forward probabilities, and each earlier state is sampled conditional on the state drawn immediately after it. This produces an exact draw from $p(\mathbf{z} \mid \mathbf{y}, \mu, \nu, \pi)$.

Because games are independent sequences, hidden states reset at the start of each game, with no carry-over between starts. FFBS is run separately for each of the G games per iteration.

Step 2: Sample transition probabilities (ν). Given the sampled state sequences, we count how many times each state-to-state transition occurs across all games. These counts, added to the Dirichlet prior parameters, form the posterior hyperparameters for each row of ν . Each row is then drawn independently from its Dirichlet full conditional.

Step 3: Sample initial state distribution (π). We count how many games begin in each state (i.e., look at the first sampled state of each game). Adding these counts to the Dirichlet prior on π gives the posterior hyperparameters, from which we draw a new π .

Step 4: Sample emission means (μ_k). Given the state assignments for every pitch, we pool all pitches assigned to state k across all games and compute their sample mean and count. Combined with the Normal prior, this gives a Normal full conditional for μ_k , from which we draw directly. This step is repeated for each state.

We also apply a **label-switching correction** after each iteration. Because the states $\{1, 2\}$ are exchangeable under the likelihood, the sampler can freely permute their labels between iterations. Without correction, samples from different chains would not be comparable. We fix this by enforcing a canonical ordering: after each draw, we sort the states by their emission means ($\mu_1 < \mu_2$), relabeling all state assignments, both rows and columns of ν , and π accordingly.

To assess convergence and reduce dependence on starting values, we run **four independent chains** from different initializations and discard the first half of each chain as burn-in. We reshape the pitch-level data into Y , a list of length G where Y_g is the ordered vector of four-seam fastball velocities for game g . We also fix σ to the empirical standard deviation of all 2024 velocities, a shared, known variance assumed by the model.

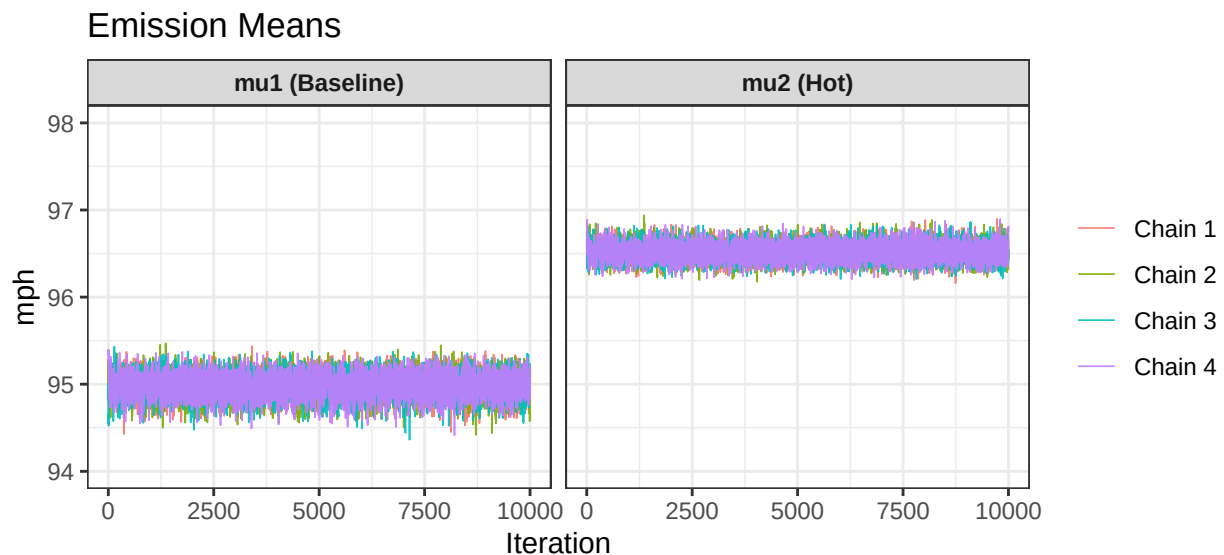
MCMC Diagnostics

We ran four independent Gibbs chains from different starting values to check convergence. Trace plots show the sampled value of each parameter at every iteration. Four well-mixed chains that overlap and wander around the same region, rather than drifting apart or getting stuck, are the primary visual signal that the sampler has converged.

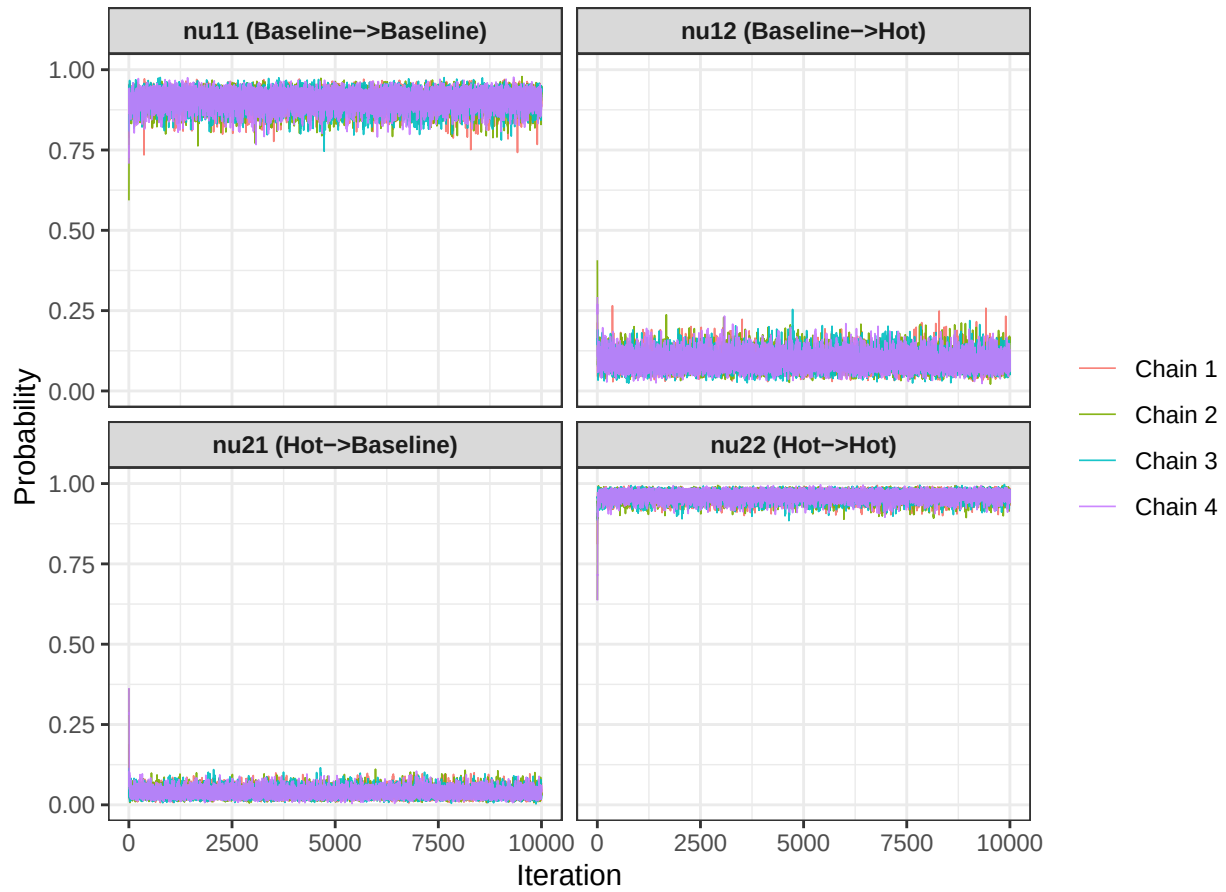
We monitor all sampled scalar parameters across three figures: emission means (μ_1, μ_2), all four transition probabilities ($\nu_{11}, \nu_{12}, \nu_{21}, \nu_{22}$), and the initial state distribution (π_1, π_2).

Initial Samples

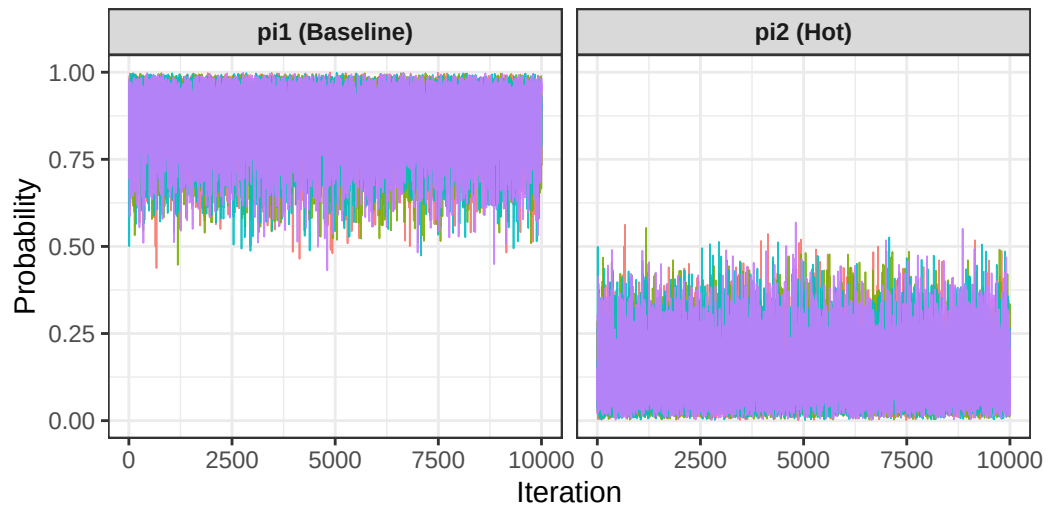
These are the raw trace plots from the Gibbs sampler: every draw from all 10,000 iterations, with no post-processing applied. Consecutive samples from a Gibbs chain are not independent: each draw depends on the one before it, so the chain can exhibit autocorrelation, where successive values move slowly and the effective number of independent samples is much smaller than the raw count.



Transition Probabilities

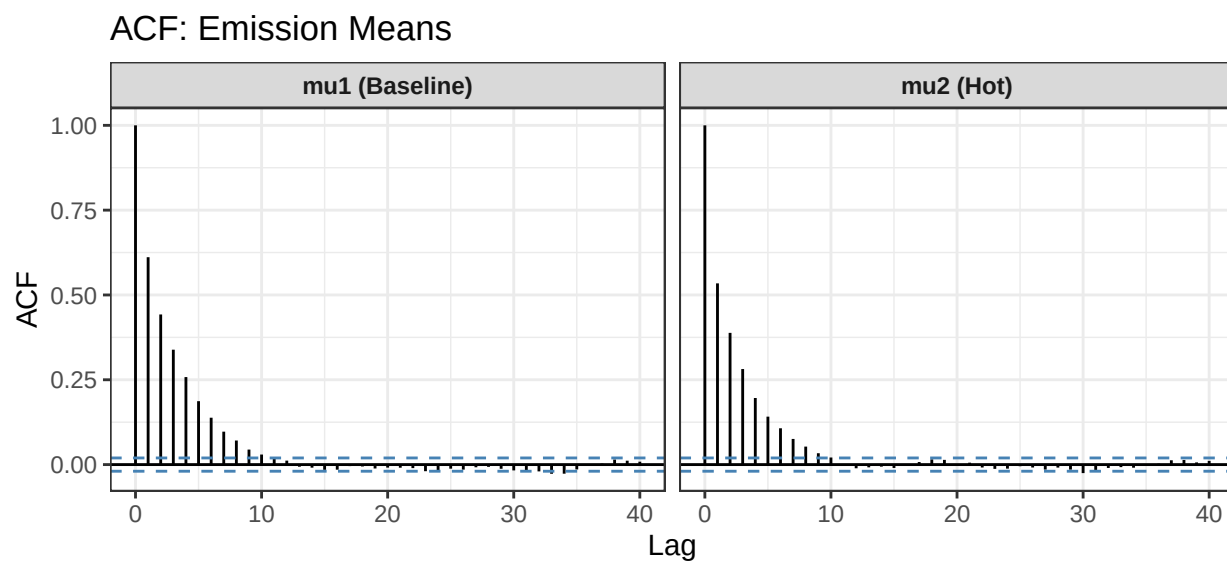


Initial State Distribution

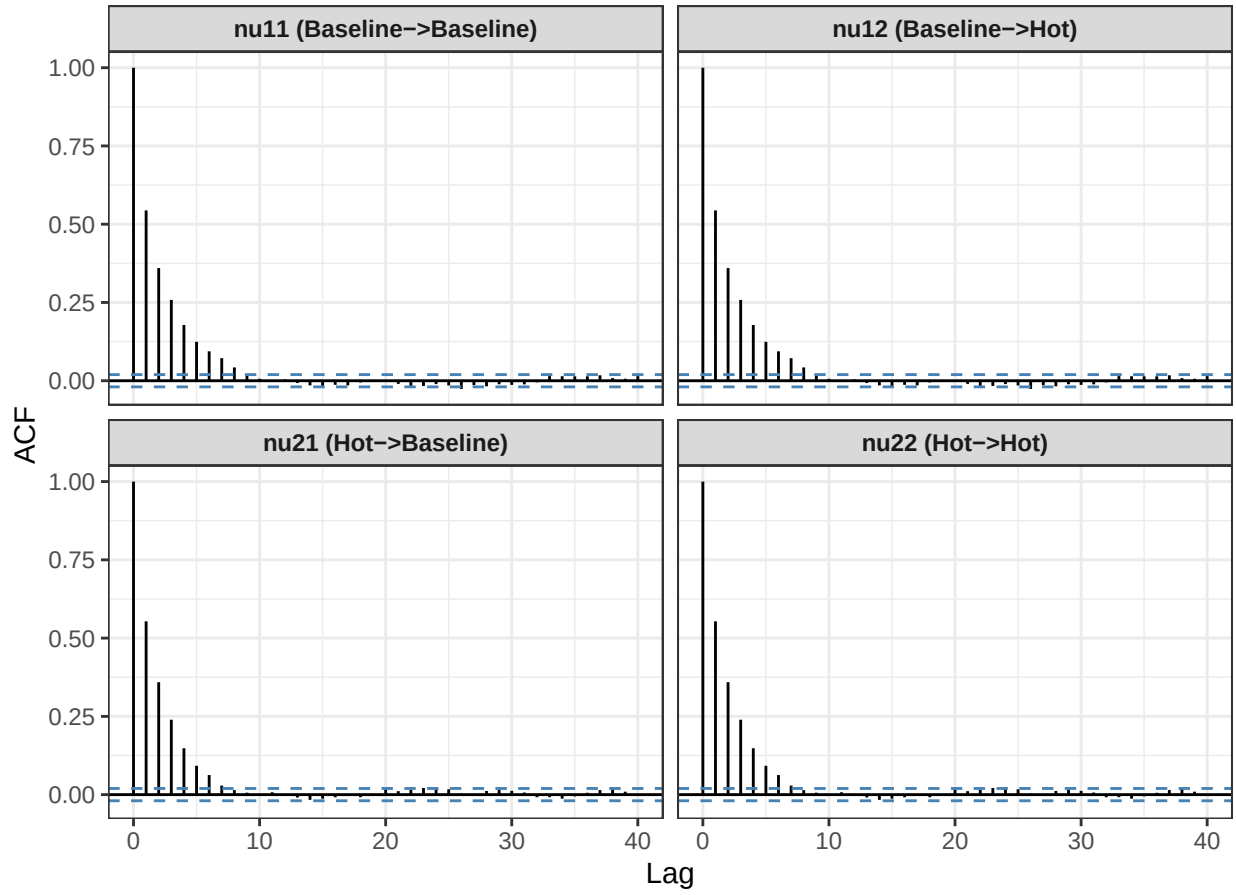


Thinned Samples

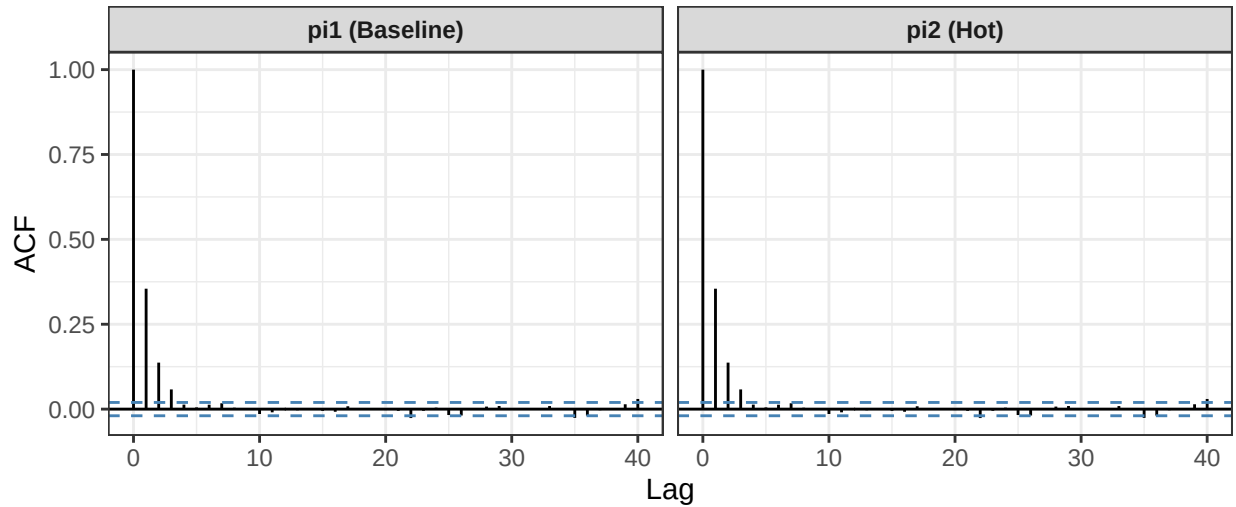
To diagnose autocorrelation, we compute the **autocorrelation function (ACF)** for each parameter. The ACF measures the correlation between a chain's samples at lag k , that is, how similar a draw is to the draw k steps before it. High autocorrelation at moderate lags (say, lags 1–10) indicates that the chain is moving slowly through the posterior and that many consecutive samples carry nearly the same information. The lag at which the ACF first drops close to zero tells us the minimum spacing needed for approximately independent draws.



ACF: Transition Probabilities



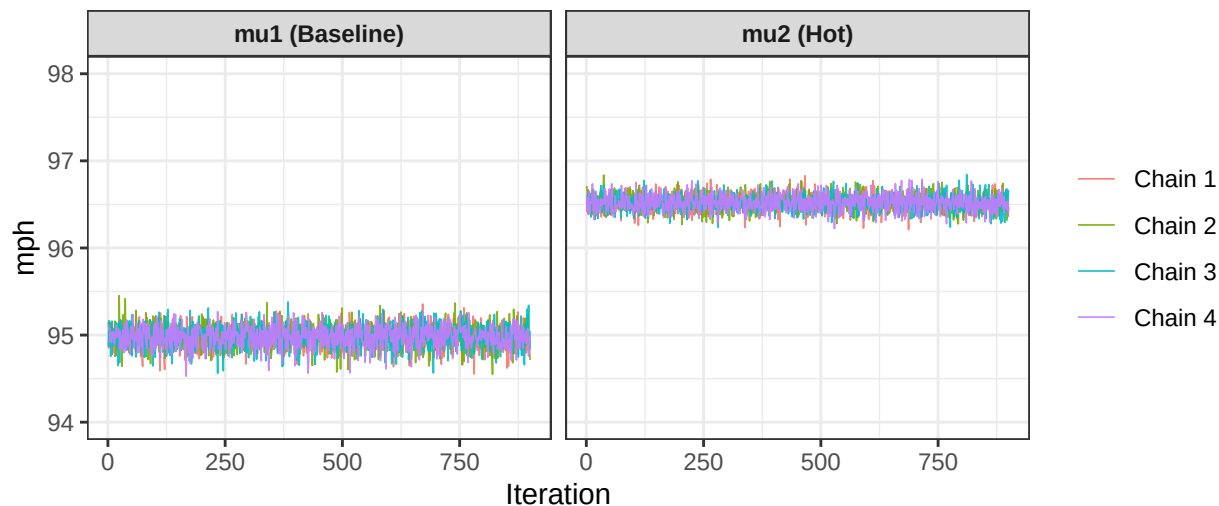
ACF: Initial State Distribution



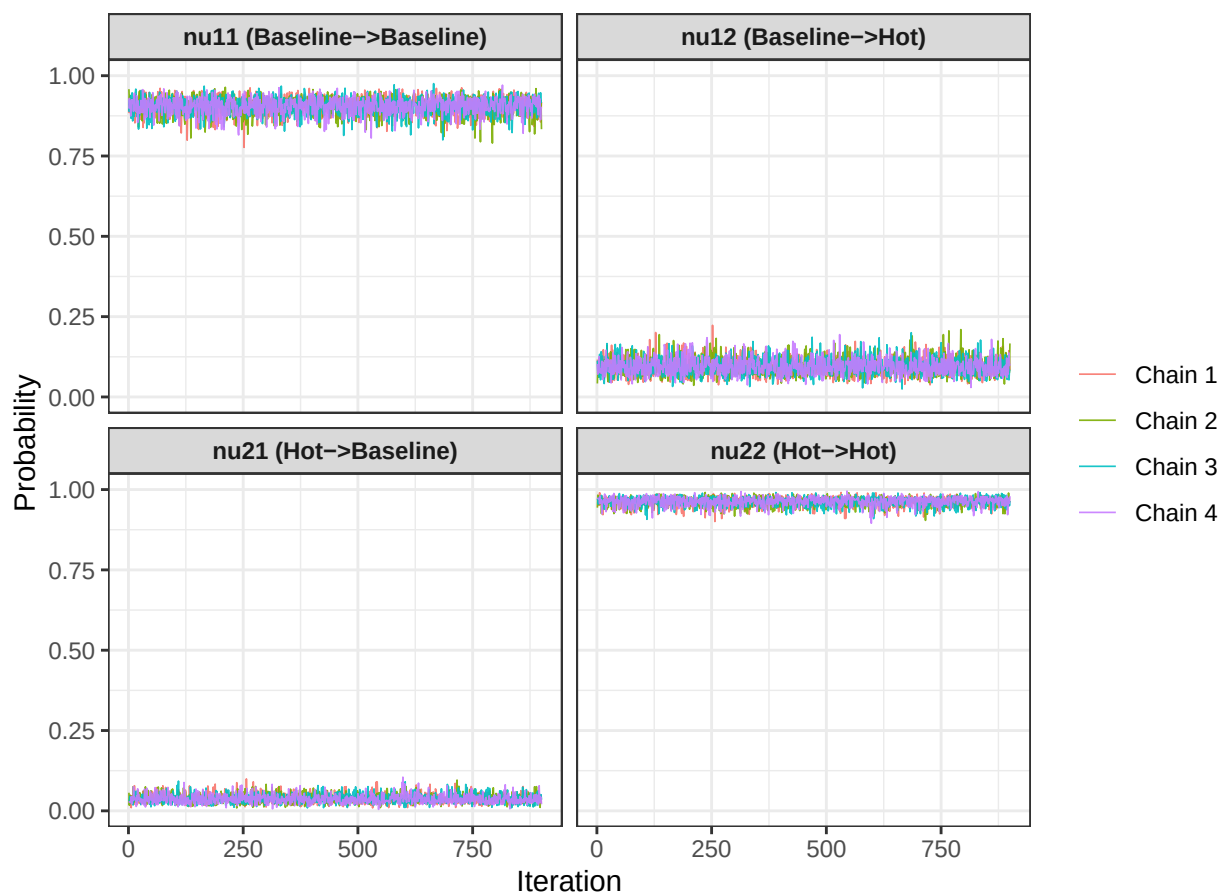
Based on the ACF, we thin the chain by keeping only every k -th sample, where k is chosen to be at or beyond the lag where autocorrelation becomes negligible. Thinning reduces the effective sample size but produces a set of draws that are much closer to independent, which makes posterior summaries and credible intervals more reliable. The trace plots below show the sampled values after thinning.

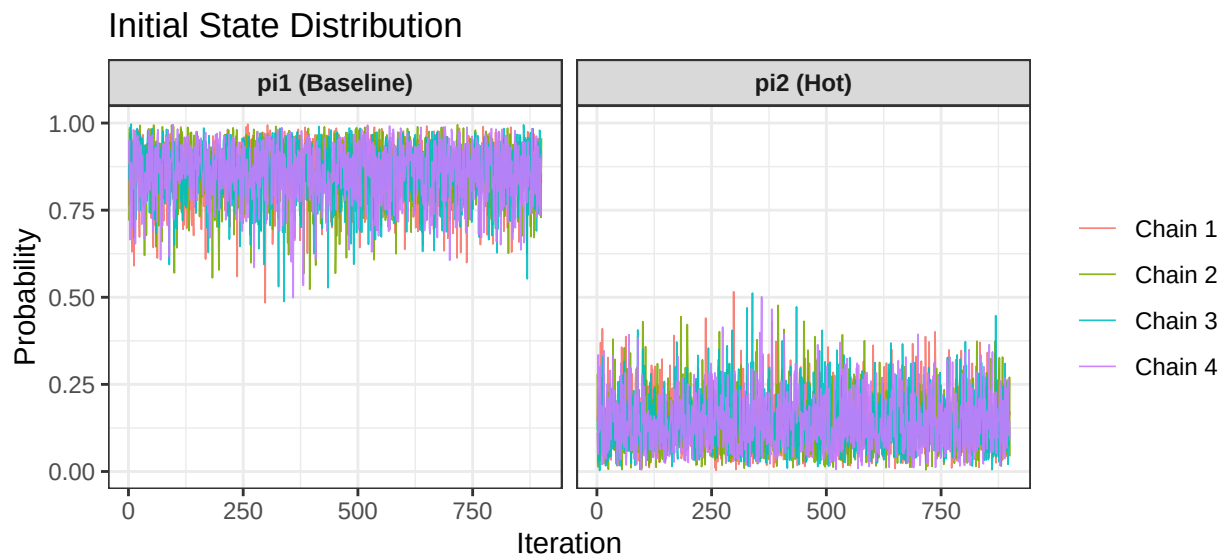
The ACF plots above show that autocorrelation across all parameters decays fully by roughly lag 8–10. We discard the first 1,000 iterations as burn-in to ensure the starting values have no influence on inference, then thin by keeping every 10th sample. This leaves 900 draws per chain (3,600 combined), which is enough for stable posterior summaries.

Emission Means



Transition Probabilities





Posterior Summaries

Table 1: Posterior summaries from 3,600 thinned draws (4 chains $\times$ 900 samples). CI bounds are the 10th and 90th percentiles.

Parameter	Mean	SD	10%	90%
mu1 (Baseline mean, mph)	94.984	0.126	94.821	95.139
mu2 (Hot mean, mph)	96.517	0.092	96.401	96.638
nu11 (Baseline $\rightarrow$ Baseline)	0.905	0.027	0.870	0.937
nu12 (Baseline $\rightarrow$ Hot)	0.095	0.027	0.063	0.130
nu21 (Hot $\rightarrow$ Baseline)	0.038	0.014	0.022	0.056
nu22 (Hot $\rightarrow$ Hot)	0.962	0.014	0.944	0.978
pi1 (Pr. start Baseline)	0.860	0.082	0.744	0.956
pi2 (Pr. start Hot)	0.140	0.082	0.044	0.256

Posterior Predictive Checks

Visualizations

4. Interpretation

What Did We Learn?

Limitations

When Should We Use This Model?

Sensitivity to Modeling Assumptions